

Case Study Documentation : Zero Retention Fake News Detector

Course : Data Structures and Algorithms

Instructor : Dr.Rithwik

Evaluator : Sriram

Date : September 30,2025

Team Members : CSE B - Group – 8

Snehitha – CB.SC.U4CSE24108 (Application and data implementor)

Siri - CB.SC.U4CSE24120 (Hashing and Bit Array Implementation)

Varna -CB.SC.U4CSE24156 (Checking and Analysis)

Dharani - CB.SC.U4CSE24161 (Add/Check Implementation in Bloom filters)

1 . Project Overview & Objectives

Zero Retention Bloom—Fake News Detector is a research-grade tool designed to efficiently identify fake news articles using Bloom filters. By scanning for suspicious language, clickbait, unreliable sources, and other risk signals, the system offers rapid and memory-efficient detection suitable for large-scale, real-time use.

Objectives:

- Detect fake news quickly and with minimal memory usage using probabilistic filters.
- Provide a multi-dimensional risk analysis (e.g., emotional manipulation, medical claims, bias) for each article.
- Enable scalable news screening with zero false negatives and controllable false positives.
- Support user analysis, demonstration tests, and comprehensive filter statistics for educational and practical applications.

2 . Team Collaboration and Process

Meeting 1: Initial Design

- **Discussion :** The First meeting is all about on defining the project scope and selecting appropriate real world problem of bloom filters data structure . we selected zero retention fake news dectector due to in recent years, With the explosive growth of online content and social media, fake news has become a major threat to public awareness and information integrity. Conventional fake news detection systems often rely on storing large datasets and performing heavy computations, which raise privacy, storage, and scalability concerns

Meeting 2: Design & Architecture

- Finalized the modular system structure: separate classes for filtering logic, analysis, and user interface.
- Chose and customized hash functions (Murmur, FNV-1a, DJB2) for the filter.
- Planned multi-filter design covering suspicious language, source reliability, and other dimensions.

Meeting 3: Implementation & Integration

- Integrated all code modules; cross-tested filter insert/query, UI, and demo routines.
- Reviewed data structure analysis for documentation.
- Ran group tests on demonstration cases (medical, bias, satire, emotional manipulation).

Meeting 4: Testing, Review & Enhancement

- Group code review for robustness, edge cases, and filter efficiency.
- Brainstormed and documented additional features: visual outputs, advanced analytics, and memory usage comparisons.

3.System Architecture and Design

3.1 Proposed Solution:

To overcome these limitations, we aim to implement a Zero Retention Fake News Detector that uses Bloom Filters Data Structure which is a probabilistic and memory efficient data structure to verify the authenticity of news without storing the original data permanently. This implies real-time membership checks against a database of known to be fake or verified news patterns while ensuring zero data retention, fast lookup, and minimal memory usage. This enhances user privacy and allows for efficient large-scale fake news detection in resource-constrained environments.

3.2 Why Bloom Filters ??

Traditional data structures such as arrays, lists, hash tables, or trees can store and search data accurately, but they have major drawbacks when applied to fake news detection that must process millions of entries quickly and efficiently which is can't done other data structures.

Bloom Filters are preferred because they offer speed, memory efficiency, and privacy, which are essential and appropriate for a Zero Retention Fake News Detector.

3.3 How it Works ??

A Bloom Filter is a probabilistic data structure that quickly checks whether an element belongs to a set using minimal memory. It consists of a bit array, where all bits start as 0, and multiple hash functions that map each item to specific bit positions. When inserting data, bits at those positions are set to 1. During lookup, if all corresponding bits are

1, the item is probably present, if any bit is 0, it is definitely not present. This method ensures fast, memory-efficient, and zero-retention detection.

Bit Array: Core memory structure storing membership bits and initialized to all zeros.

Hash Functions: Several independent functions use hash algorithms providing uniform mapping of fingerprints across the bit array.

Counting Bloom Filter : This allowing removal by incrementing and decrementing counters instead of raw bits—handy for real-world scenarios with evolving news databases

3.4 Key Properties of Bloom Filter :

1. Memory Efficiency:

Bloom Filters require very little memory to represent large sets of data. Unlike hash tables or arrays that store entire entries, a Bloom Filter only stores bit patterns based on hash values.

2. High-Speed Lookup :

Bloom Filters provide $O(k)$ lookup time (where k is number of hash functions), which is extremely fast.

3. Privacy-Friendly (Zero Retention) :

Since Bloom Filters don't store the actual data, only hashed representations, they preserve user privacy and meet the "zero retention" requirement.

4. Scalability :

As data grows (millions of fake news items), other data structures like arrays or trees become inefficient. Bloom Filters handle massive datasets easily with minimal increase in size.

3.5 Bloom Filter Management:

Initialization:

Allocates a predetermined bit array and configures multiple hash functions according to desired false-positive rate and expected amount of articles.

Insertion:

For each known fake news fingerprint, runs hash functions and sets corresponding bits in the array to 1.

Zero Retention Module:

Periodically flushes or reinitializes the Bloom filter so only recent or most dangerous signatures are retained, reducing long-term error accumulation.

3.6 Bloom Filter Analysis :

Structure	Space Complexity	Time Complexity	Pros	Cons
Bloom Filter	$O(m)$ bits	$O(k)$	Memory efficiency, speed, simplicity	False positives, no deletions
Counting Bloom Filter	$O(m \cdot \log(n))$ bits	$O(k)$	Deletion possible, still compact	Higher memory than basic Bloom
Hash Table	$O(n)$	$O(1)$	Deterministic, no false positives	High memory, slower for large data
BST/Trie	$O(n)$	$O(\log n)$	Ordered, searchable	Slow, not optimal

3.7 Analysis of Implementation :

Modules :

1. BloomFilterBitArrayResearch

- **Constructor:** Calculates optimal bit array size, instantiates BitSet, and outputs initialization stats.
- **Hash Functions:** hashMurmur, hashFNV1a, hashDJB2. Each takes a string (item) and seed, returning an index into the bit array.
- **add:** Given a string input, sets bit indices for all 3 hash results.
- **check:** Using 3 hash results, confirms whether all relevant bits are set or not
- **printStatistics:** Displays total bits, filled bits, empty bits, fill rate, and memory usage.

2. FakeNewsDetector

- **trainFilters:** Populates each Bloom filter using curated lists for suspicious keywords, clickbait phrases, known fake domains, etc.
- **analyze:** Evaluates article by checking words, phrases, source URL, author, and more against all filters; returns analysis report.
- **Check Functions:** Each component (emotion, clickbait, bias, medical, etc.) uses its dedicated filter for detection.
- **Heuristics:** Adds extra signals (all-caps, excessive punctuation, odd URL structure) and adjusts risk score accordingly.
- **printFilterStatistics :** Calls each filter's stats for memory and occupancy analysis.

3. NewsAnalysis

- **Score calculation and risk level:** Assigns risk using the sum of all detected patterns, applies thresholds for HIGH/MEDIUM/LOW risk.
- **Verdict:** Returns a short conclusion (“Likely Fake News”, “Verify Before Sharing”, “Appears Credible”).
- **Formatted Report:** Outputs full diagnostic, sample matched patterns, and suggestions.

4. Bloom Filter Fake News Detector :

- **Interactive Loop:** Lets users analyze their own articles, run demos, or see filter statistics.
- **analyzeUserArticle:** Accepts and processes user input through the detector.
- **runDemonstrationTests:** Executes scripted tests demonstrating medical misinformation, credible news, bias, satire, and emotional manipulation analysis.

5. Other Functionality :

- **hasFactCheckingLanguage:** Checks for scholarly phrases (“studies show”, “peer reviewed”).
- **calculateCredibilityScore:** Heuristic scoring based on analytical keywords in the content.
- **hasUrgencyManipulation:** Flags urgent/clickbait headline constructions.
- **hasProperSourcing:** Looks for citations and responsible journalism.

Criteria	Bloom Filter	Hash Map
Memory usage	2.8 KB for 9 filters	50-100 KB or more
Lookup time	Constant ($O(1)$)	$O(1)$
False Positives	Yes (tunable)	No (exact)
False Negatives	Never	Never
Deletion	No	Yes

3.8 Faces Difficulty :

- Multi-dimensional scoring provides transparent flags (“political bias”, “emotional manipulation”, etc.).
- Human-readable output ensures accessibility in public or academic domains.

4.Complexity Analysis & Justification

4.1 Time Complexity

- **Hash functions (MurmurHash, FNV-1a, DJB2):** Each runs in linear time with respect to the length n of the input string, i.e., $O(n)$.
- **Add and Check operations in Bloom filters:** Constant time $O(k)$, where $k=3$ is the fixed number of hash functions.
- **Article Analysis:** Linear in the number of words/sentences w , overall $O(w \cdot k \cdot f)$ where $f=9$ is the number of Bloom filters.
- **Training filters:** Linear in the number of training items times number of hash functions, i.e., $O(n \cdot k)$.

4.2 Space Complexity

- **Each Bloom filter:** $O(m)$ bits, where m is the bit array size optimized based on expected elements and false positive rate.
- **Total Bloom filters:** $9 \times m$, extremely memory efficient compared to storing full patterns.

- **Auxiliary structures:** Minimal additional space for lists and flags, negligible compared to Bloom filters.

4.3 Justification of Efficiency

- Bloom filters ensure very fast membership tests $O(1)$ with minimal memory footprint, key for real-time, large-scale fake news detection.
- Controlled false positive rate guarantees no false negatives, so no fake news is missed; acceptable false positives are manageable by deeper analysis.
- The bit-array size and hash count are mathematically optimized according to desired false-positive targets ensuring best trade-off between memory and accuracy.
- Compared to exact set storage (hash tables), Bloom filters reduce memory by over 90%, a critical advantage when processing millions of news items continuously.

4.4 Why Optimal ??

Bloom filters are optimal for fake news detection because they use very little memory, super fast, and guarantee all known fake news patterns are detected with zero false negatives—perfect for large-scale, real-time screening

5 . Conclusion

The Zero Retention Bloom Fake News Detector efficiently uses Bloom filters to rapidly and accurately flag fake news with minimal memory use. Its probabilistic approach guarantees no fake news is missed while keeping false positives low. This combination of speed, space efficiency, and reliability makes it an excellent solution for real-time large-scale detection, demonstrating strong potential for practical deployment and further enhancement.

6.References

[A BLOOM FILTERS BASED DATA MANAGEMENT WITH ERROR DETECTION AND CORRECTION](#)

<https://proceedings.neurips.cc/paper/2020/file/86b94dae7c6517ec1ac767fd2c136580-Paper.pdf>

<https://www.sciencedirect.com/science/article/pii/S2667096821000446>

https://en.wikipedia.org/wiki/Bloom_filter

<https://www.geeksforgeeks.org/advance-java/bloom-filters-in-java-using-redis/>

<https://github.com/barrust/bloom/blob/master/src/bloom.c>